The Last Page: Seeded Navigation and Time-Dependent World Erasure

System Design and Implementation Report

October 2026

ABProjects

***Abstract*—The Last Page is a self-contained 3D browser game about traversing a drawn environment before the page disappears. Its challenge is generated from a player-visible seed, a selected spawn and exit gate, a navigable region graph, and a schedule of staged environmental erasure. The game combines a bundled Three.js renderer, an embedded Rapier physics/WebAssembly runtime, character movement and camera control, path and feasibility analysis, dynamic visual and physical erasure, procedural audio, and a run controller with menu, pause, result and creative modes. This report describes the implemented architecture and the correctness problem at its center: a generated route must initially be traversable and remain meaningfully playable for its assigned timer, while browser focus changes must not consume challenge time without player control. It explains the scenario-selection and validation pipeline, distinguishes visible erasure from physical removal, and examines event ordering, deterministic seeds, frame scheduling and deployment as a single file. A focused regression test demonstrates a fix to the original focus-loss behavior; rendering while paused or hidden is also reduced without altering active traversal. The available checks are explicit and limited: they establish syntax, startup, a begun timed run and the focus-loss timer invariant, but do not establish full seed-sweep feasibility or hardware performance.**

***Index Terms*—procedural scenarios; deterministic generation; navigation graph; timed traversal; state machine; physics; WebGL; browser lifecycle.**

I. Introduction

The Last Page uses an unusual failure condition: the traversable world itself progressively vanishes. A stick-figure player must reach a gate to the next page before routes are erased. This changes the ordinary relationship between game world and game clock. In a static maze, a path that exists at the start remains feasible unless actors block it. In The Last Page, an initially navigable path can become impossible as stages of erasure affect visual surfaces and eventually physical traversal. A meaningful challenge therefore requires not only a route from start to goal but a route compatible with a time-dependent map.

The runtime combines a drawn 3D environment, a character controller, multiple gameplay modes and a navigation-challenge layer. The page also contains embedded libraries and assets, including Three.js and the Rapier WASM physics runtime. The delivered file remains a single `index.html` so it can be opened or deployed as an HTML5 itch.io upload without additional runtime files. That packaging property should not be confused with a trivial implementation: large parts of the document are embedded graphics, physics and media data, while the actual game logic contains a substantial navigation and erasure controller.

Three problems shape the design. First, seed-based level selection must provide reproducible, valid spawn and gate scenarios. Second, time must advance only in appropriate play states. Third, the visual effect of disappearing paper must agree with collision and route viability. The technical analysis below follows those three constraints and treats responsive input, presentation and failure handling as supporting systems.

II. Architecture and execution model

A. Embedded engine and navigation controller

The runtime contains 3D scene construction, renderer and camera, collision world, a player movement/interaction layer, sound and presentation effects. A higher-level controller, constructed by `PWCreateNavigationGame`, manages seed normalization, scenario selection, gating, erasure, HUD, menus, scoring, pause/resume and results. It exposes a public diagnostic API, including a scenario snapshot facility and a batch seed checker. The architecture diagram is conceptual: the distributed program still contains these subsystems in one physical HTML file.

The game state distinguishes startup/menu, active, paused and terminal phases, with creative behavior treated separately from a timed navigation run. The world and controller communicate through explicit hooks: placing a gate, settling the player, hiding or revealing layers, and updating HUD/camera state. A state transition may need to change input lock, audio playback and display layers together. A screen merely *looking paused* is not enough if the underlying elapsed timer continues to increment.

B. Physics and visual world

The embedded Rapier setup controls colliders and character/world interaction; the source also contains a fixed-step physics accumulator and a 60 Hz step configuration for its collision world. Three.js renders surfaces, sketched geometry, environment effects and camera views. Physics colliders and renderable meshes therefore have related but nonidentical

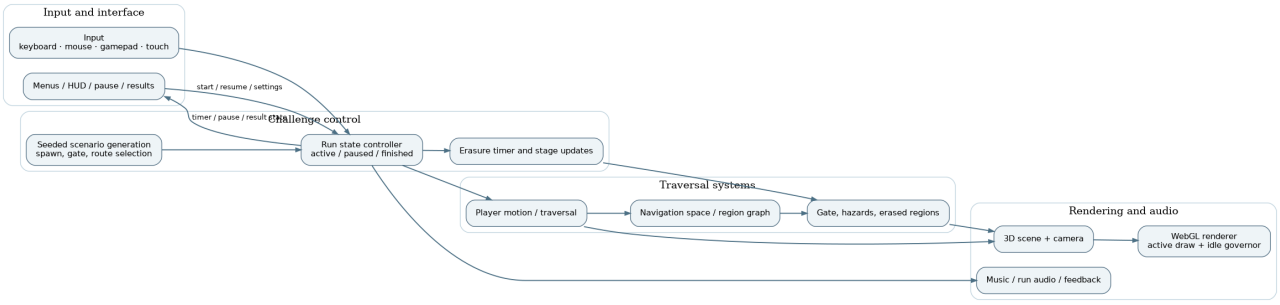


Fig. 1. Functional architecture of The Last Page. Seeded generation selects a challenge whose timed erasure alters route viability; the run controller governs input, physics, presentation and audio.

representations. Removing a mesh from view does not necessarily remove its collider, while disabling a collider does not necessarily hide its corresponding mesh.

That separation is particularly important here because erasure is both an artistic effect and a traversal mechanic. The navigation layer has to coordinate visible stage changes, collision removal and reachability. The camera follows character motion but must also remain responsive during menu/pause transitions. Both WebGL and physics may be expensive; neither is made more correct by simply reducing the other system's update frequency.

III. Seeded scenario generation

A. Seed normalization and pseudo-random choice

The player can enter or generate a seed. The controller normalizes that input and derives a deterministic pseudo-random stream. Candidate route choices use seeded shuffling rather than uncontrolled `Math.random` calls inside the selection loop. Given the same stable world graph, seed, code revision and generation configuration, the intended invariant is

$$G(s, \mathcal{W}, \theta) = G(s, \mathcal{W}, \theta),$$

where G returns a normalized scenario description, s is the seed, $\mathcal{W}$ the authored world/navigation data, and θ the generation parameters. The identity is intentionally simple: its practical content is that hidden time, global RNG or previous runs must not contaminate scenario choice. Reproducibility of generated descriptions does not imply that every physics run will finish identically under variable input timing.

The source's `scenarioSnapshot` records a reproducible summary, and `testSeeds` compares repeated generation for the same seed. This makes deterministic-generation checks part of the product rather than an afterthought. A seed printed in the UI can serve as a concrete, shareable test case for a difficult route.

B. Navigation graph and candidate structure

The game indexes traversable spaces into regions and edges. Its routines include `connect`, `dijkstra`, `routeOptions`, `regionGraphDistances`, and route profiling. A candidate scenario comprises a spawn location, a gate location, a route or region-hop estimate, separation information, and a demand score used in balancing. Spatial clearance, slope and local validity are also checked, rather than assuming that every point in a region is safe to stand on.

For an edge-weighted graph $\mathcal{G} = (V, E)$, a shortest-path distance between nodes u and v is

$$D(u, v) = \min_{p: u \rightsquigarrow v} \sum_{e \in p} w(e).$$

A small Euclidean separation does not guarantee a short playable route. Walls, stairs, separated floors or a missing connection may force a much longer path. Conversely, a distant gate may be directly accessible through an open corridor. The source's region-hop and route-profile measures therefore complement physical distance. These estimates support balancing and feasibility screening but are not a proof of how fast a human can traverse the world.

C. Scenario validation

A candidate needs several distinct checks. The spawn must be stable and clear of forbidden geometry. The destination gate must be placeable, visible or discoverable as intended, and reachable by a valid route. The candidate must fit the challenge's timer and regional erasure order well enough not to create an impossible level. Functions such as `locationIsClear`, `groundSlope`, `spawnCanSeeGate`, `profileRoute`, `makeSchedule`, `routeSurvivesSchedule`, `validateScenario`, and `generateScenario` reflect this sequence.

A useful representation is to treat a scenario as a tuple

$$S = (s, p_{\text{spawn}}, p_{\text{gate}}, R, \sigma, T),$$

where R describes the planned route, σ is the scheduled erasure ordering, and T is the permitted run time. Validation then asks whether the static path is legal and whether a plausible time-ordered traversal can remain ahead of physical erasure. This is a constrained generation problem, not merely random placement of two objects.

A seed sweep should report rates of candidate rejection, path-length and regional-hop distributions, generation failures and repeated-seed disagreements. The source provides test facilities for those quantities, but this report does not invent a numerical success rate from a sweep that was not executed in the required compatible browser environment.



Fig. 2. Deterministic scenario selection: route and erasure constraints are screened before the run begins.

D. Candidate rejection and failure taxonomy

Not every seed/candidate should be accepted. A sensible rejection taxonomy distinguishes an invalid local spawn from an unreachable gate, an insufficiently clear landing area, a route profile outside the allowed demand range, and an erasure schedule that invalidates the path too early. These are different failure modes with different remedies. If the candidate generator repeatedly chooses unreachable pairs, changing the erasure timer cannot repair the graph. If a valid static route fails only after regional deletion, the order or timing of erasure requires attention instead. Keeping those categories separate avoids hiding a topology error behind more generous difficulty settings.

The source's seed testing facility can evaluate these cases without altering the game itself. The procedure is to generate a scenario from a seed, serialize a normalized snapshot, generate it again under the same code and world state, and compare the snapshots. A mismatch indicates that deterministic input is being contaminated by hidden state or nondeterministic ordering. Validated scenarios can then be aggregated by number of regional hops, route separation, difficulty or erasure timing. Such a sweep is a more meaningful quality metric than a handful of successful runs on specially chosen seeds, but it must actually be executed before numerical rates are reported.

A deterministic generator also provides a debugging advantage: a player-reported seed can be turned into a reproducible test case. A failed report should retain its seed, expected gate position, route region sequence, any rejected candidates and the first erasure stage that made a route invalid. This information can be recorded externally for inspection; the playable delivery can remain one HTML file.

IV. Time-dependent erasure model

A. Stages and physical threshold

The source declares five erasure stages with an eighteen-second stage duration, alongside a threshold for physical removal. Continuous stage progress drives visuals, while an integer stage or threshold controls collision/world removal. These stages let a region fade and change appearance before it becomes physically absent. The distinction gives players warning of impending loss and makes the disappearance visually meaningful.

Let t be elapsed active run time and $\sigma(v)$ the configured erasure schedule offset for region v. A conceptual stage function is

$$q_v(t) = \text{clamp}\left(\frac{t - \sigma(v)}{18}, 0, 5\right).$$

This illustrates how a region's staged progress can be parameterized by elapsed time; the real source contains its own stage mapping and must be regarded as authoritative for exact boundary behavior. The visual stage may use the fractional part of q_v, while physical validity uses a discrete condition such as $\lfloor q_v \rfloor \geq k_{\text{physical}}$. The controller includes routines `regionStageFloat`, `applyRegionStage`, `updateErasure`, `isPhysicalVoidAt`, and `isColliderErased` for related responsibilities.

These functions should agree on the erasure boundary. If a renderer makes ground disappear before its collider is disabled, the player may seem to stand on air; if collision vanishes much earlier than the presentation indicates, the player can fall through apparently solid paper. The game deliberately separates visual and physical state, so the alignment between them must be regression-tested.

B. Route viability over time

An ordinary navigation edge may be traversable at $t = 0$ but not at $t > t_e$ if its region is erased. The correct question is therefore not only whether a static path exists, but whether there exists a time-indexed route that crosses each necessary region while it remains viable. Conceptually, a feasible route $R = (v_0, \ldots, v_m)$ with arrival times τ_i satisfies

$$\tau_0 = 0, \quad \tau_{i+1} \geq \tau_i + \delta(v_i, v_{i+1}),$$

$$\text{traversable}(v_i, \tau_i) = \text{true} \quad \text{for all } i,$$

where δ is an estimate of movement time or minimum traversal duration. The code's schedule and route checks approximate the relevant feasibility constraints. Human detours, failed jumps and camera mistakes remain outside an idealized path estimate, which is why a validated candidate still needs practical playtesting.

V. Traversal, input and feedback

A. Movement and interaction

The character supports movement, camera orientation, jumping, sliding and interactions. Desktop keyboard and mouse controls are complemented by gamepad processing and on-screen touch affordances. The physics runtime supplies collision and world response, while authored gameplay state determines when a jump, slide or gate interaction is permitted. The camera follows the avatar and can adjust to world geometry, but needs to avoid disorienting the player during rapid state changes or gate approaches.

The objective's legibility depends on the combination of world silhouette, gate placement, HUD and audio. The HUD shows remaining time or run progress. Visual erasure and sound reinforce urgency without altering the source of truth for the timer. Because this is a traversal challenge, a poorly behaved camera is more than a presentational weakness: it

can make a nominally reachable route difficult to execute. This suggests explicit camera and input-device tests under low ceilings, corners and moving camera targets.

B. Audio and scene effects

Music, event feedback and environmental effects are embedded in the file or generated locally. The run controller updates audio alongside movement and erasure progress; pause/resume must also change the audio state. Browsers typically require user gestures to start or resume audio contexts. The existence of a pause state therefore does not mean sound will automatically resume without user interaction in every browser. The implementation uses the existing controller transitions rather than creating a second, unrelated audio pause path.

VI. Run-state correctness

A. The clock invariant

The most important runtime invariant is that a timed run must advance only while the player can act. Let $e(t)$ be elapsed game time and Δt a clamped frame interval. Conceptually,

$$e_{n+1} = \begin{cases} e_n + \Delta t, & \text{if state is active and timer enabled,} \\ e_n, & \text{if paused, hidden or terminal.} \end{cases}$$

The inspected original code already stopped ordinary game updates when the controller entered `paused`. The discovered bug was that losing focus did not always *cause* that state transition; clearing keyboard input alone did not prevent the active erasure clock from advancing. This is why changing the timer arithmetic would have addressed the symptom in the wrong place.

The revision routes `blur` and `visibilitychange` events to the controller's existing `pause()` method when appropriate. That method is idempotent with respect to the active-state check: delivering both events need not trigger two incompatible pause transitions. The transition also handles pointer lock, motion/audio state and the visible pause interface. After a deliberate resume action, normal updates may continue. Correctness here comes from one owner for the run-state transition.

B. Focus-loss regression observation

A targeted browser experiment began a timed run and dispatched a focus-loss event in both source versions. In the original, the navigation controller remained active and elapsed time increased from approximately 0.165 to 0.365 seconds during the observation window. In the revised build, the state became `paused` and elapsed time remained approximately 0.1353 seconds. This supports the narrow claim that the missing focus-loss transition was fixed in the tested configuration. It is not an exhaustive test of all browser tab-management policies or device suspend states.

C. Outcomes and scoring

The navigation controller has distinct success, failure and result paths. Finishing a run should stop the timer, determine the score once and transition to results. Losing to erasure should similarly enter a failure phase and disallow further authoritative movement. The source includes separate routines for preparing a successful gate reveal and beginning/finalizing failure; their division reflects the need to stage visual feedback without allowing ordinary play to continue indefinitely.

VII. Rendering and single-file deployment

A. Background rendering

An active run updates physics and draws its scene on the normal browser callback. During a paused or completed run, a backdrop can remain visible but need not redraw at full monitor frequency. The revision limits non-active WebGL rendering to approximately 10 Hz and avoids hidden-tab draws. It does not change the number of player physics steps in active play, the erasure constants, the scenario generator or the authored world.

The source uses `requestAnimationFrame`; frame deltas are clamped, with a maximum of 100 ms for the primary frame. The clamp prevents a returning tab from initiating an unbounded single-frame simulation catch-up. Rendering less in paused mode is not equivalent to slowing erasure: game time is governed by the state transition described above, not the graphics refresh interval.

B. Packaging and source integrity

The released HTML is about 5 MB, because it embeds rendering, physics/WASM and media resources. That size is consistent with an intentional offline and single-file design. Splitting these resources into fetched files could improve source organization in another project, but would violate the required distribution model and introduce additional caching and network failure modes. The technical documentation remains external to the running program and does not impose module imports.

The repository records original and revised SHA-256 hashes in `docs/RELEASE_MANIFEST.json`, together with a human-readable change summary. These hashes support exact artifact identification but cannot prove behavior from source equality alone; for instance, a single new event handler can have large effects at runtime. The separate `tests/verify.mjs` checks parser validity and expected structure, not a full game playthrough.

VIII. Verification and validity

A serious verification suite should test at least: repeated normalized seeds; generated route connectivity and clearance; spawn and gate placement; representative multi-region traversal; early/late erasure collision agreement; a successful gate arrival; timer expiry and failure; pause/resume; tab hide/return; and gamepad/touch input. For performance, log frame-time percentiles, physics step count, collision query time and WebGL draw calls on real hardware. The existing

Verification activity	Evidence obtained	Important limit
Self-contained source inspection	Three inline JavaScript blocks parse and expected embedded resources remain.	Syntax does not prove runtime behavior.
Browser startup	Software-WebGL Chromium displayed the menu and started one timed scenario.	Not a full route-to-gate completion.
Focus-loss A/B check	Active clock advanced in baseline; patched state paused and clock stopped.	Tested a controlled blur event, not all lifecycle cases.
Seed-validation facilities	Built-in `testSeeds`/snapshot functions can check generated scenarios.	No large empirical sweep result is claimed.
GPU performance	Non-active renderer throttling is present in source.	No measured GPU time or active-run speedup.

TABLE I
Source-grounded verification and acceptance evidence. A listed test obligation is not an assertion that it passed.

`docs/VALIDATION.md` provides an acceptance checklist but is not itself evidence those cases all passed.

A. An end-to-end seeded-run trace

A complete run can be reviewed as a sequence of state changes rather than as a single visual success. A seed is normalized and candidate locations are enumerated; a route profile and schedule are calculated; validation selects one scenario; world regions and colliders reset to their initial state; the spawn and gate are placed; the player's input becomes active; and the clock begins. Every subsequent frame updates a bounded time increment, player/controller state and the visible/physical erasure state associated with the run. A gate interaction produces a success transition, while timeout or physical failure enters the appropriate losing path. Only terminal-state logic should commit the result.

This sequence illustrates why reusing a stateful world across runs is delicate. The second run must not inherit a region that was deleted in the first, a disabled collider that has not been restored, a stale gate marker, or elapsed time left in an old controller. Seed determinism is meaningful only when the initial world snapshot is equivalent. A useful repeatability test should run the same seed after a different seed and compare both its scenario snapshot and the initial region/collider states.

B. Priority among lifecycle transitions

Multiple events may arrive nearly together: a player reaches the gate as the timer expires, a browser tab is hidden during a failure animation, or a pause gesture occurs while the result screen is opening. An orderly controller needs a clear priority among terminal outcome, active simulation and presentation transitions. Once a win or loss has been committed, a late input or blur handler should not reopen an active run or award a second result. The corrected focus-loss behavior delegates to the existing controller precisely to avoid an independent, competing notion of pause.

Because the game uses separate physics, renderer, audio and DOM components, the safest test checks all four layers after a transition. For example, after a pause: the clock is unchanged, player motion is stopped, audio is paused or correctly gated, and the pause UI is interactive. After success: the gate result appears, the timer no longer advances, and no simulation callback can move the player into an erased region behind the result screen. These are directly testable invariants and are more informative than a screenshot of a visible "Paused" label.

IX. Limitations and conclusion

The scenario validator is a practical feasibility filter, not a proof that every human player can finish every selected seed. Route cost estimates omit some control difficulty and camera factors; time-dependent erasure introduces additional failure boundaries. Reproducible scenario descriptions do not guarantee identical physics outcomes under different frame/input histories. The single-file build supports straightforward deployment but makes dependency review more difficult. Finally, the available browser checks cover startup and a focused timing regression rather than comprehensive traversal correctness or hardware graphics benchmarking.

The engineering identity of The Last Page lies in coupling deterministic scenario generation to a time-varying navigable world. Its controller must keep visual erasure, physical collision, generated route feasibility, user action and run time consistent. The source-level focus-loss fix restores a key timer invariant, and background-render gating reduces avoidable drawing without changing the active challenge. These are defensible improvements precisely because they respect the separation between game state and presentation.

References

[1] ABProjects, *The Last Page*, shipped `index.html`, source/release revisions recorded in `docs/RELEASE_MANIFEST.json`.
[2] ABProjects, *The Last Page*, itch.io game page, gameplay premise and public release.
[3] ABProjects, `tests/verify.mjs` and `docs/VALIDATION.md`, packaging checks and manual acceptance plan.
[4] Three.js, *JavaScript 3D library*, embedded rendering runtime, threejs.org.
[5] Dimforge, *Rapier physics engine*, embedded WASM runtime and API, rapier.rs.